```
}
@end
```

Given below is the code for HelloWorldApp.m:

```
#import "HelloWorldApp.h"

@implementation HelloWorldApp
- (void) applicationDidFinishLaunching: (id) unused
{
  UIWindow *window;

  window = [[UIWindow alloc] initWithFrame:[[UIScreen mainScreen]
bounds]];

  /* Create a text view */
  textView = [[UITextView alloc]
    initWithFrame: CGRectMake(0.0f, 48.0f, 320.0f, 100.0f)];
  [textView setText:@"Ciao"];

  /* Create a main view, and add our text view as subview */
  mainView = [[UIView alloc] initWithFrame: [[UIScreen mainScreen] bounds]];
  [mainView addSubview:textView];

  /* Setup window */
  [window makeKeyAndVisible];
  [window addSubview: mainView];
}
@end
```

The following is the code for Main.m:

```
#import <Foundation/Foundation.h>
#import <UIKit/UIKit.h>
#import "HelloWorldApp.h"

int main(int argc, char **argv) {
  int retval;

  NSAutoreleasePool *pool = [ [ NSAutoreleasePool alloc ] init ];
  retval = UIApplicationMain(argc, argv, @"HelloWorldApp", @"HelloWorldApp");
  [pool release];
  return retval;
}
```

To build this application, you first have to make sure that cross compilers are in the path:

```
$ export PATH=$PATH:/toolchain/pre/bin
```

Then run the following command:

```
$ arm-apple-darwin9-gcc -o HelloWorld Main.m -lobjc \
-framework CoreFoundation -framework Foundation \
-march=armv6 -mcpu=arm1176jzf-s
```

Here is the explanation of the above command:

- ***arm-apple-darwin9-gcc*** is the name of the cross-compiler itself. This is located in */itoolchain/pre/bin*.
- ***-o HelloWorld*** tells the compiler to output the compiled executable to a file named HelloWorld.
- ***Main.m*** is the name of the source file(s) being included in the program, separated by spaces. The ***.m*** extension tells the compiler that the sources are written in Objective-C.
- ***-lobjc*** tells the compiler to link in the tool chain's Objective-C messaging library, which is needed by all iPhone applications.
- ***-framework CoreFoundation -framework Foundation*** are two of the base frameworks to be linked into the application. Depending on what components of the operating system are being used in the code, different frameworks provide different functionality.
- ***-march=armv6 -mcpu=arm1176jzf-s*** sets the correct architecture and CPU type of the executable.

Now create the iPhone .app (Application) file:

```
$cp -p HelloWorld ./HelloWorld.app/
```

Figure 6: The HelloWorld app –yes, it does seem dumb!

Once the application is built, you can deploy it on your iPhone using ***scp*** command:

```
$ scp -r HelloWorld.app root@<ip address of iphone>:/Applications
```

Sign your application (required for firmware >= 2.0) by running the following command from the iPhone SSH terminal:

```
$ ldid -S /Applications/HelloWorld.app/HelloWorld
```

Go to the iPhone Spring Board (iPhone App launcher interface) and launch your application. Your application will look something like the screenshot in Figure 5.

Sure I know that's a dumb application, but go check out the App Store and find out what's possible. END

By: Kunal Deo

The author is a veteran open source developer. He's currently leading two open source projects – WinOpen64 & KUN Wiki. He has contributed to many projects including, KDE-Solaris, OpenMoko and Python mw-serve. Kunal has written numerous articles on FOSS, Solaris and Linux-related technologies for various technical magazines around the globe. He is also writing a book titled "Porting On Open Solaris". In his free time he loves playing games on his XBOX360 and Playstation3. He blogs at *kunaldeo.blogspot.com*.